POLYTECH
NICE SOPHIA

Collections et possession

Présentation : Tom Niget

Mail de contact : polytech@niget.fr

Site et ressources : niget.fr

Cours réalisé grâce notamment aux ressources de :
Julien Deantoni, Benjamin Werner, Joseph Albahari

Tableaux et vecteurs

- En C, une fonction qui reçoit un tableau doit aussi recevoir sa **taille**, dans un paramètre séparé (cf. TD 2)
- Rien ne vérifie que les deux correspondent
- `std::vector` (`<vector>`) garde ensemble ses éléments et leur nombre, et peut **s'agrandir**

Tableau C

```
int total(const int values[],
          std::size_t count);

int sizes_kb[] = {12, 300, 4};
int sum = total(sizes_kb, 3);
```

`std::vector`

```
int total(const std::vector<int>& values);

std::vector<int> sizes_kb = {12, 300, 4};
sizes_kb.push_back(80);
int sum = total(sizes_kb);
std::cout << sizes_kb.size(); // 4
std::cout << sizes_kb[1];     // 300
```

Usage et comportement

```cpp
int total(const std::vector<int>& values) {
    int sum = 0;
    for (int value : values) {
        sum += value;
    }
    return sum;
}
```

Passage par référence constante : un passage par valeur copierait tous les éléments du vecteur

La boucle parcourt les éléments dans l'ordre, comme pour un tableau

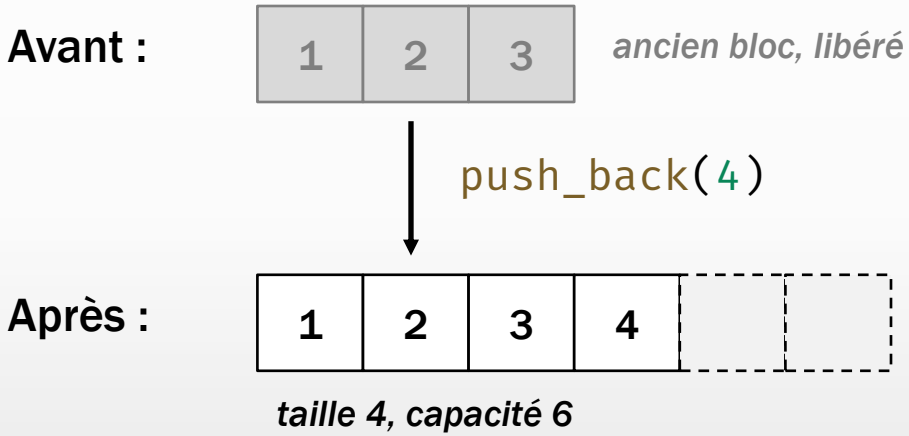
- `values[i]` ne vérifie pas l'indice ; `values.at(i)` le vérifie et signale l'erreur (exceptions : séance 4)
- `std::array<int, 3>` (`<array>`) remplace un tableau C de taille fixe : même stockage, sans allocation, mais il connaît sa taille et se copie comme une variable ordinaire

Stockage des éléments

- Les éléments sont rangés **côte à côte**, dans un bloc alloué dynamiquement
- Bloc plein : `push_back` en alloue un plus grand, y **déplace** les éléments et libère l'ancien

```
std::vector<int> values = {1, 2, 3};
// size() == 3, capacity() == 3

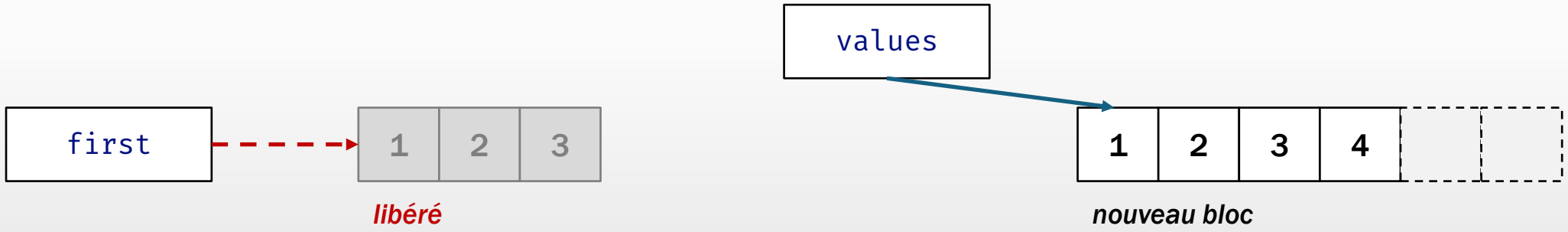
values.push_back(4);
// size() == 4, capacity() == 6 avec GCC
```



- `reserve(n)` prépare de la place pour n éléments, sans en créer
- À sa destruction, le vecteur détruit ses éléments et libère son bloc : c'est du **RAII**, comme `unique_ptr` et `std::string`

Pointeurs vers des éléments / réallocation

```cpp
std::vector<int> values = {1, 2, 3};
int* first = &values[0];
values.push_back(4);  // peut changer de bloc
std::cout << *first;  // comportement indéfini si c'est le cas
```



Le compilateur ne dit rien : comme au CM 1, un pointeur invalide compile très bien.

Exemple : système de fichiers

- Fichiers et dossiers ont un **nom** et une **taille**
- Classe mère abstraite `Entry`, sur le modèle de `Component` au CM 2

hérite de ←

```cpp
class Entry {
public:
    virtual ~Entry() = default;
    const std::string& name() const;
    virtual int size_kb() const = 0;

protected:
    Entry(std::string name);

private:
    std::string name_;
};
```

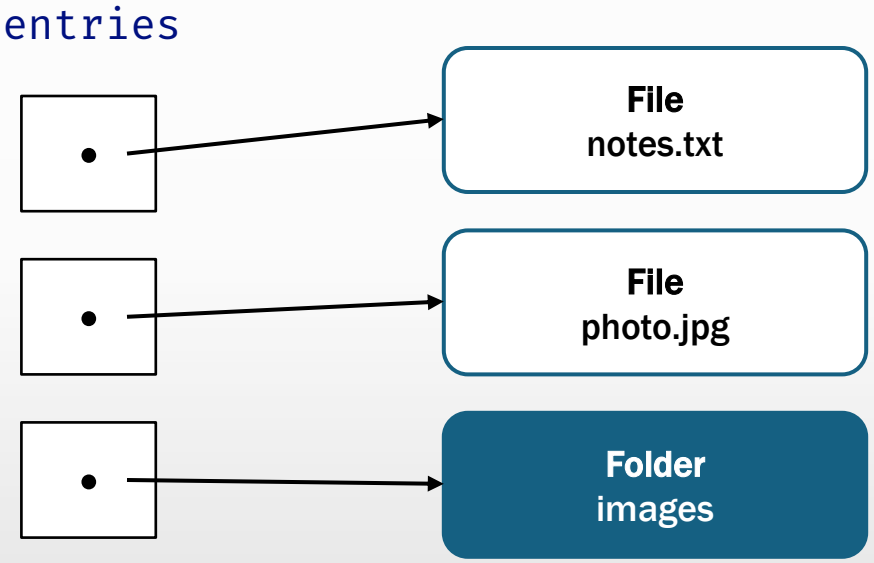
```cpp
class File : public Entry {
public:
    File(std::string name, int size_kb);
    int size_kb() const override;

private:
    int size_kb_;
};
```

Ranger des objets de types différents

```
std::vector<Entry> entries;
entries.push_back(File("notes.txt", 12)); // ne compile pas
```

- `Entry` est **abstraite**, ça forcerait le vecteur à construire des `Entry`
- Avec une classe mère concrète : **slicing**, on ne garderait que la partie `Entry` de chaque fichier et tout usage des infos contenus serait illicite
- On stocke donc des **pointeurs** : chaque objet garde son type dynamique



Reste à savoir qui détruira les objets pointés.

Posséder ou observer

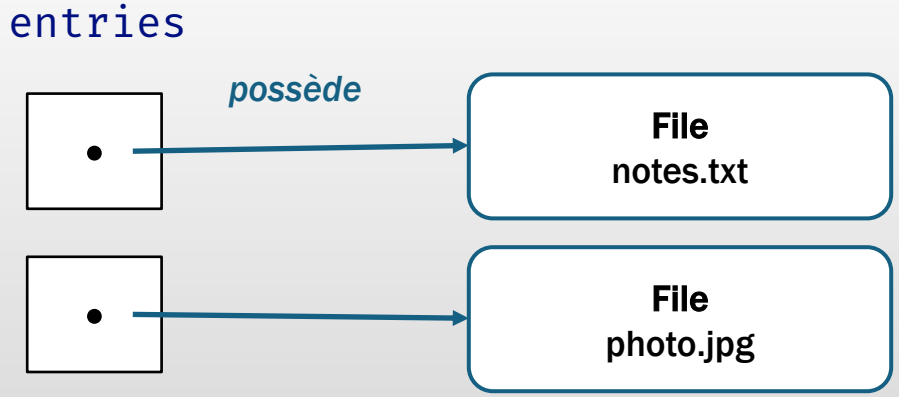
Celui qui a la charge de détruire un objet le **possède**. Les autres peuvent l'**observer** : ils y accèdent, sans impacter sa durée de vie.

Écriture	Rôle
variable locale, champ stocké par valeur	possédé par le bloc ou par l'objet qui le contient
`std::unique_ptr<T>`	**possède** l'objet pointé
`T&`, `const T&`	**observe** un objet
`T*`, `const T*`	**observe** un objet, ou rien (`nullptr`)

Vecteur possédant les éléments

```cpp
std::vector<std::unique_ptr<Entry>> entries;
entries.push_back(std::make_unique<File>("notes.txt", 12));
entries.push_back(std::make_unique<File>("photo.jpg", 2400));

for (const auto& entry : entries) {
    std::cout << entry->name() << " : " << entry->size_kb() << " ko\n";
}
```



- `auto` déduit le type de l'élément ; `&` évite de le copier (de toute façon illicite pour un `unique_ptr`)
- Le vecteur **possède** ses fichiers : ils sont détruits avec lui

Observateur qui survit à l'objet

```
std::vector<const File*> recent;
{
    File notes("notes.txt", 12);
    recent.push_back(&notes);
} // notes est détruit
std::cout << recent[0]->name(); // pointeur pendant
```

Un raccourci vers un fichier supprimé ne mène nulle part.
En C++, ce code compile, et l'accès a un **comportement indéfini**.

- `recent` **observe** des fichiers rangés ailleurs : il ne doit plus servir une fois ces fichiers détruits
- Une fonction qui reçoit `const File&` observe aussi, mais sans risque : l'appelant garde le fichier en vie jusqu'au retour de la fonction

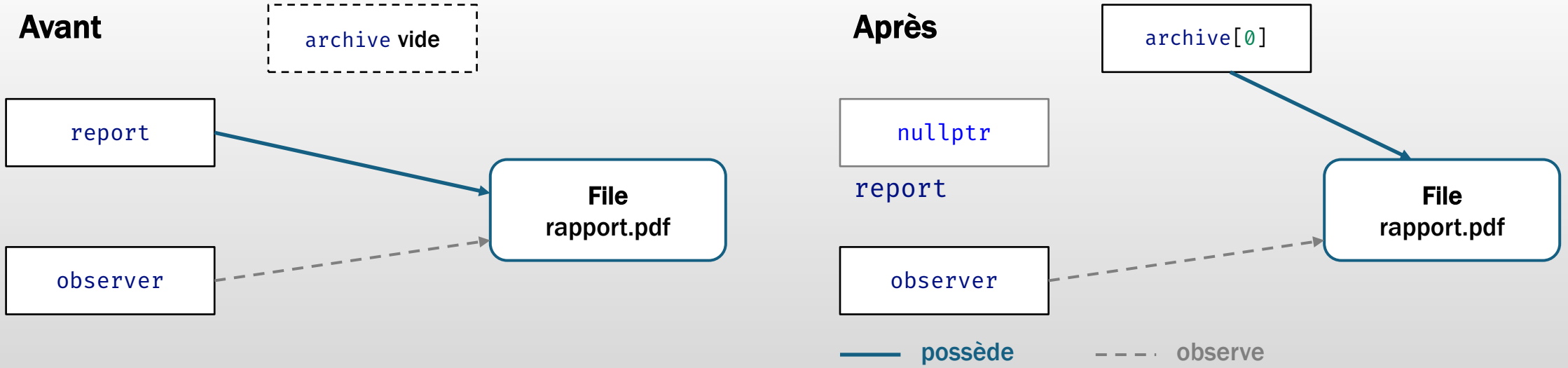
Transférer la possession

- Un `unique_ptr` ne se copie pas : deux propriétaires détruiraient deux fois le même objet
- On peut en revanche **transférer** l'objet à un autre propriétaire

```cpp
auto report = std::make_unique<File>("rapport.pdf", 250);
File* observer = report.get();

std::vector<std::unique_ptr<Entry>> archive;
archive.push_back(std::move(report)); // <utility>
```

`std::move` ne déplace rien à lui seul : il autorise `push_back` à vider `report`.



Possession et fonctions

La signature d'une fonction indique ce qu'elle fait de l'objet :

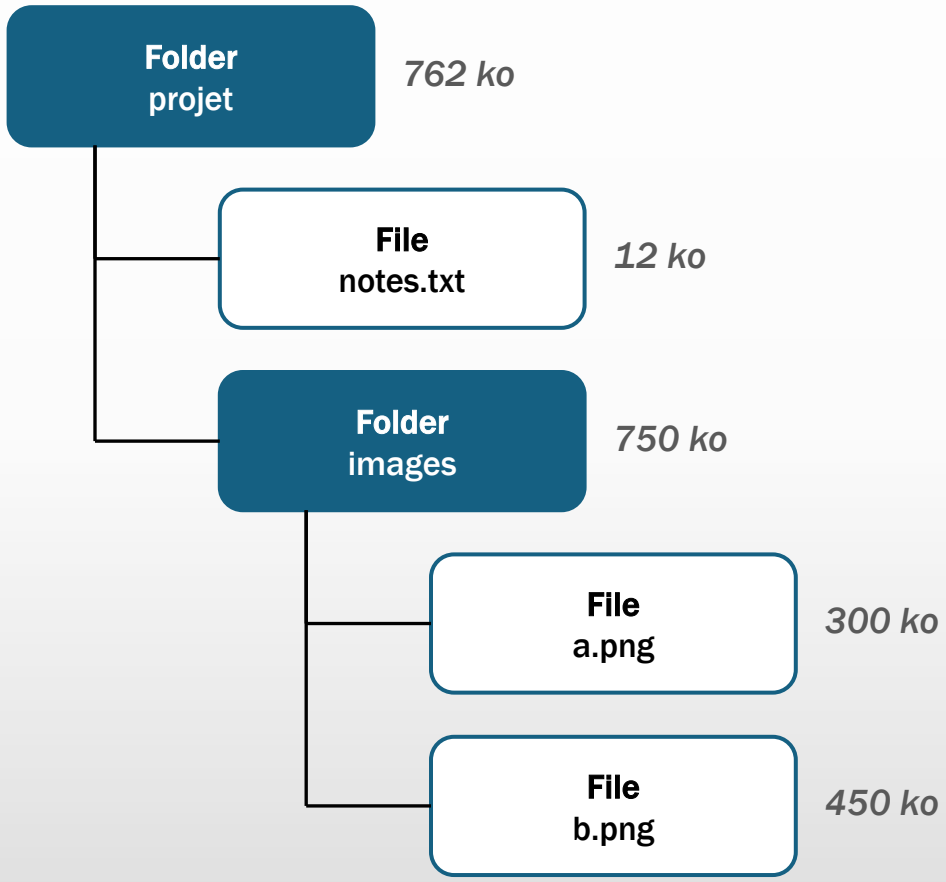
```
void print(const Entry& entry);                    // observe pendant l'appel
void move_to_trash(std::unique_ptr<Entry> entry); // prend possession
std::unique_ptr<Entry> make_readme();              // crée et cède la possession
```

```
auto readme = make_readme();       // readme possède le fichier
print(*readme);                    // prêté le temps de l'appel
move_to_trash(std::move(readme)); // cédé : readme vaut nullptr
```

- Dans `make_readme` : `return std::make_unique<File>("README.md", 2);`
- Une variable locale renvoyée par `return` n'a pas besoin de `std::move` : elle va disparaître, le compilateur la déplace

Un pointeur obtenu avec `get()` observe : on ne s'en sert jamais pour `delete`. L'objet a déjà un propriétaire, qui le détruira de son côté.

Un arbre d'objets




```cpp
class Folder : public Entry {
public:
    Folder(std::string name);
    void add(std::unique_ptr<Entry> entry);
    int size_kb() const override; // somme des tailles

private:
    std::vector<std::unique_ptr<Entry>> entries_;
};
```

Un dossier est une `Entry` : il peut être rangé dans un autre dossier.
`size_kb()` sur un dossier appelle celui de chaque entrée, qui descend à son tour dans les sous-dossiers : c'est le **motif composite**.

Détruire un arbre

- Le dossier **possède** son contenu : détruire `projet` détruit tout l'arbre
- Son vecteur détruit ses `unique_ptr`, qui détruisent leurs entrées, et ainsi de suite
- `Folder` n'a pas besoin de destructeur écrit à la main : ses champs s'en chargent (**règle de zéro**)
- Un `Folder` ne se copie pas : il faudrait copier ses `unique_ptr`

Ordre de destruction d'un objet :

